

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа “ЛР2: Реализация микросервисов”

Выполнил:
Березина Софья
Группа
К3339

Проверил: Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

Целью лабораторной работы являлось разделение ранее реализованного монолитного бэкенд-приложения fitness-platform на набор самостоятельных микросервисов. Требовалось перенести основные функциональные области монолита в отдельные сервисы, обеспечить их независимый запуск и придерживаться подхода database-per-service, при котором каждый сервис работает только со своей базой данных.

Ход работы

В ходе выполнения лабораторной работы монолитное приложение было разделено на шесть сервисов: auth, user, workout, userworkouts, progress и blog. Такое разделение соответствует основным доменным областям исходной фитнес-платформы: аутентификации, пользователям, тренировкам, назначенным тренировкам, прогрессу пользователя и блогу. Каждый сервис реализован как отдельное Express.js-приложение и имеет собственный Dockerfile, package.json и точку входа index.js.

Для реализации подхода database-per-service в docker-compose.yml были добавлены отдельные контейнеры PostgreSQL: auth-db, users-db, workouts-db, user-workouts-db, progress-db и blog-db. Сервис auth хранит учетные записи, email, хэши паролей и роли пользователей. Сервис user хранит пользовательские профили. Сервис workout хранит тренировки и упражнения. Сервис userworkouts хранит назначения тренировок конкретным пользователям. Сервис progress хранит записи прогресса, а blog хранит статьи блога. Прямой доступ к чужим таблицам не используется.

При старте каждый сервис подключается к своей базе данных, ожидает ее готовности и создает необходимые таблицы с помощью команд CREATE TABLE IF NOT EXISTS. Это позволяет поднять приложение командой docker compose up --build без ручной подготовки схемы БД. Для проверки работоспособности в каждом сервисе добавлен endpoint /health.

Межсервисное взаимодействие реализовано через HTTP-запросы. При регистрации auth-service создает учетную запись в собственной БД и обращается к user-service для создания профиля пользователя. Userworkouts-service перед назначением тренировки проверяет наличие пользователя через user-service и наличие тренировки через workout-service. Blog-service перед публикацией статьи проверяет существование автора через user-service.

В результате были реализованы основные endpoint'ы: регистрация, вход и проверка JWT-токена; получение и обновление профиля пользователя; создание и получение тренировок; назначение и завершение тренировки; получение и ручное добавление прогресса; создание и просмотр статей блога. Для описания HTTP-интерфейсов подготовлена OpenAPI-спецификация `microservices.openapi.yaml`.

Вывод

В результате лабораторной работы монолит `fitness-platform` был разделен на набор микросервисов с независимыми базами данных. Реализация сохраняет основную функциональность исходного приложения и демонстрирует ключевые принципы микросервисной архитектуры: разделение ответственности, обособленное хранение данных и взаимодействие сервисов через явно описанные API.